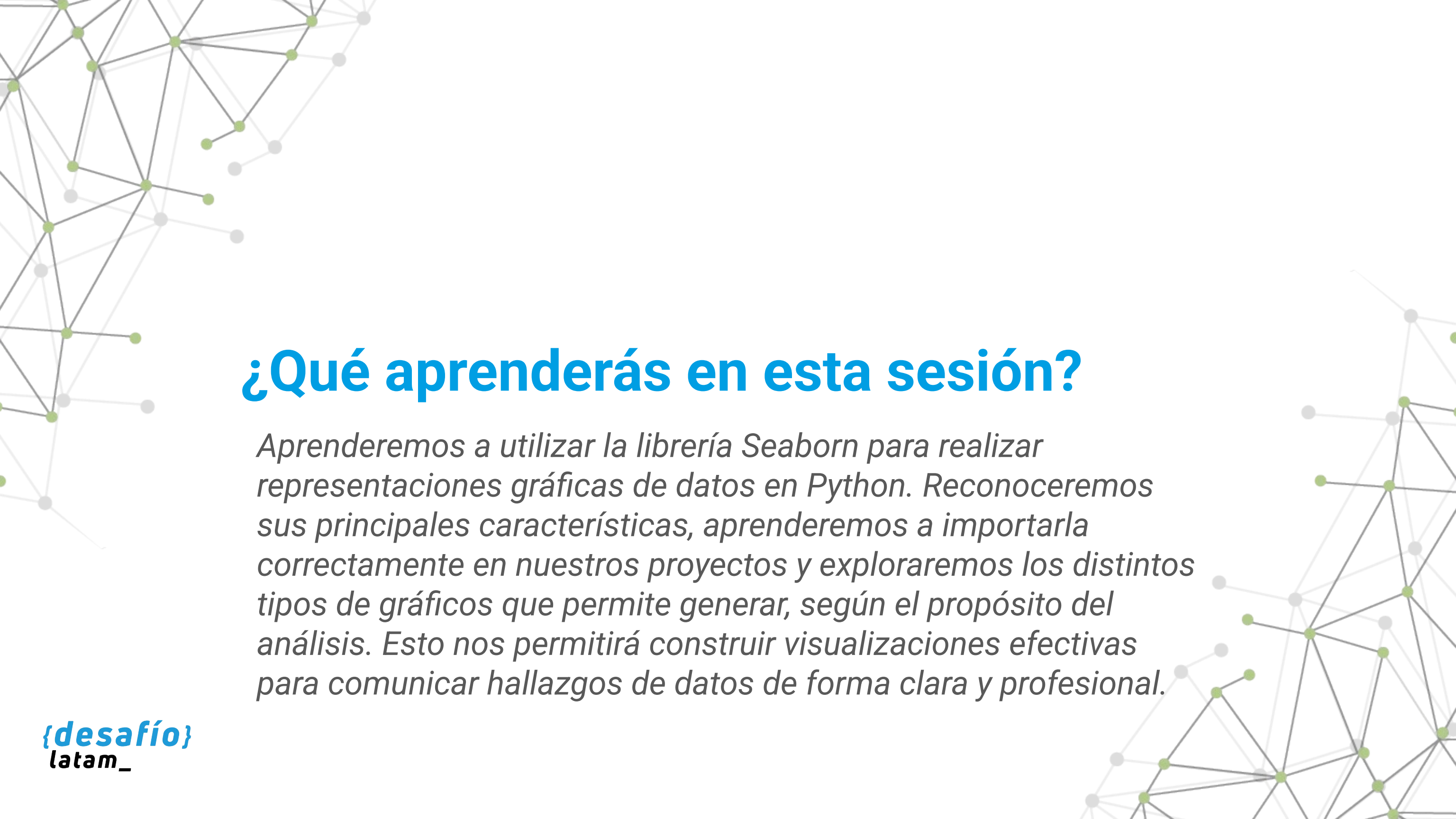




Visualización de datos

Clase 1 - Visualización de datos con Seaborn



¿Qué aprenderás en esta sesión?

Aprenderemos a utilizar la librería Seaborn para realizar representaciones gráficas de datos en Python. Reconoceremos sus principales características, aprenderemos a importarla correctamente en nuestros proyectos y exploraremos los distintos tipos de gráficos que permite generar, según el propósito del análisis. Esto nos permitirá construir visualizaciones efectivas para comunicar hallazgos de datos de forma clara y profesional.

Desafío inicial

Eres parte del equipo de inteligencia de datos de una startup de logística. El área de operaciones necesita comprender de manera visual cómo se comportan las entregas en distintas zonas geográficas, cómo varía el tiempo promedio por tipo de envío y si existen patrones estacionales en el volumen de entregas.

¿Cómo podríamos presentar estos datos de forma visual, clara y comprensible para un equipo no técnico? ¿Qué tipo de gráfico deberíamos usar para cada caso? ¿Cómo nos puede ayudar Python, y particularmente la librería Seaborn, a lograrlo?

A lo largo de esta sesión, aprenderás a utilizar Seaborn, una de las librerías más poderosas y populares de Python para visualización. Conocerás cómo se importa, qué tipos de gráficos permite construir y cómo elegir el tipo adecuado de visualización según el propósito del análisis. Esto te permitirá transformar datos en información visualmente significativa para facilitar la toma de decisiones.



/* Seaborn */

Características de la librería Seaborn

¿Qué tiene de especial Seaborn y por qué la preferimos para visualizar datos en Python?

Cuando trabajamos con datos, uno de los principales retos es comunicar hallazgos de forma visual y comprensible. Seaborn es una librería de Python basada en Matplotlib que simplifica la creación de gráficos estadísticos elegantes y con estilos predeterminados. Su gran ventaja es que ofrece funciones de alto nivel que permiten generar visualizaciones complejas con muy pocas líneas de código.



Principales características de Seaborn

1

Integración con pandas

Integra de forma nativa con DataFrames de pandas, facilitando el trabajo con estructuras tabulares.

2

Funciones estadísticas

Ofrece funciones específicas para análisis estadístico visual, como gráficos de dispersión con regresión, distribuciones, mapas de calor, entre otros.

3

Estilos predeterminados

Establece automáticamente estilos estéticos y paletas de colores agradables, lo que reduce el esfuerzo de configuración.

4

Composición de gráficos

Permite fácilmente composición de gráficos complejos (por ejemplo, FacetGrid, Pairplot) para comparar múltiples variables en paralelo.

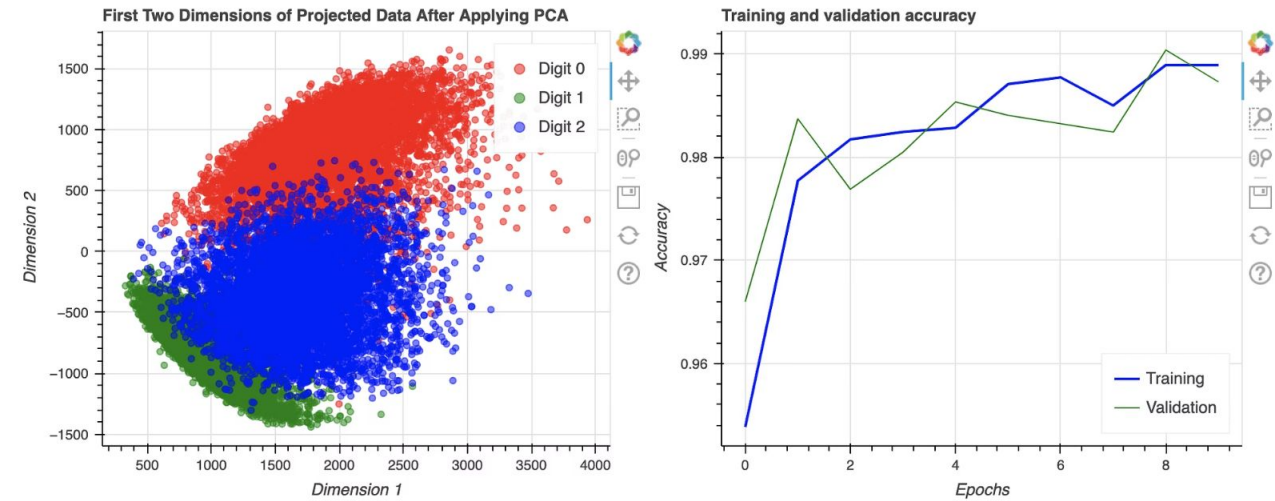
5

Sintaxis simplificada

Su sintaxis es más declarativa y simple que la de Matplotlib, lo cual la hace especialmente útil para quienes están aprendiendo.

Ejemplo comparativo

```
# Con Matplotlib:  
import matplotlib.pyplot as plt  
plt.plot([1, 2, 3], [4, 5, 6])  
plt.title("Ejemplo básico")  
plt.show()  
  
# Con Seaborn:  
import seaborn as sns  
sns.lineplot(x=[1, 2, 3], y=[4, 5, 6]).set(title="Ejemplo básico")
```



Como podemos ver, Seaborn permite lograr una visualización más limpia y con menos código.

Buenas prácticas al usar Seaborn

Antes de utilizar Seaborn, conviene tener presente algunas recomendaciones clave para garantizar que nuestras visualizaciones sean efectivas, legibles y útiles en contextos profesionales.

Importación correcta

Asegúrate de importar primero pandas y seaborn, y tener tus datos organizados como un DataFrame.

Configuración de estilo

Usa `set_theme()` al inicio para configurar un estilo visual uniforme en todos los gráficos.

Etiquetado claro

Agrega siempre títulos y etiquetas claras (eje X/Y) para que el gráfico sea autoexplicativo.

Selección adecuada

Escoge el tipo de gráfico en función del tipo de variable (categórica, numérica, temporal, etc.).

Errores comunes al utilizar Seaborn

Una mala implementación puede hacer que una visualización no cumpla su propósito o incluso induzca a conclusiones incorrectas. Estos son errores comunes que debes evitar:

-  **Gráficos inapropiados**
Utilizar gráficos inapropiados para el tipo de dato (por ejemplo, usar un boxplot para variables categóricas no ordenadas).
-  **Datos sin limpiar**
No limpiar previamente los datos (valores faltantes o mal tipados generan errores en Seaborn).
-  **Sobrecarga visual**
Sobrecargar un gráfico con demasiadas variables sin uso de facetas.
-  **Falta de leyendas**
Ignorar la leyenda o la escala de colores, dificultando la interpretación.

Como hemos visto, Seaborn ofrece una forma eficiente, estética y poderosa de construir gráficos estadísticos en Python, especialmente cuando trabajamos con datos tabulares. Para comenzar a aprovechar todas sus funcionalidades, lo primero es asegurarnos de que la librería esté correctamente instalada e importada en nuestro entorno de trabajo. En el siguiente apartado revisaremos cómo hacerlo paso a paso.

/* Utilizando Seaborn */

Importación de la librería

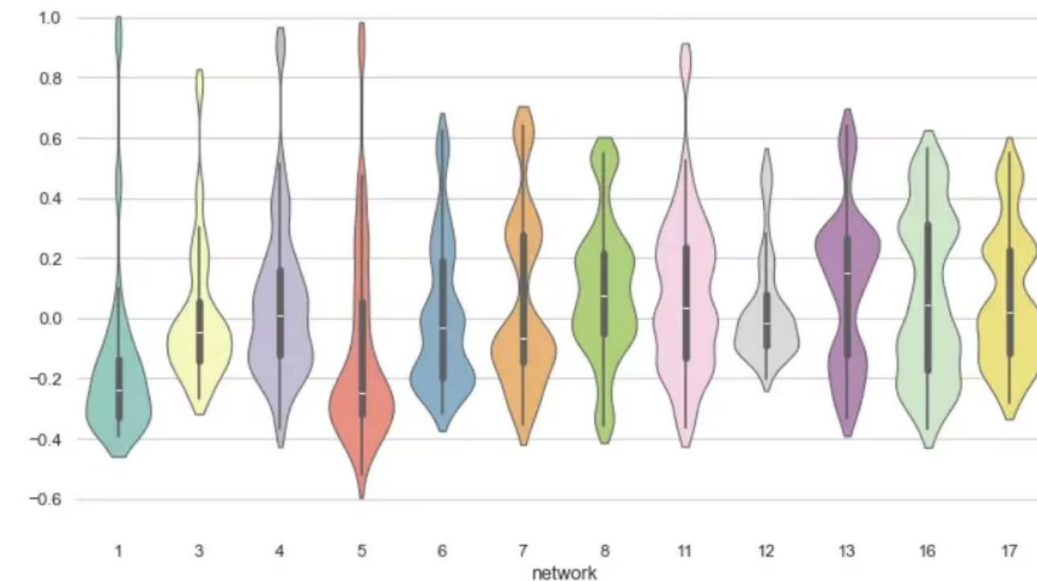
¿Qué pasos debo seguir para comenzar a trabajar con Seaborn en mi entorno de desarrollo?

Antes de usar Seaborn, debemos asegurarnos de que la librería esté instalada y correctamente importada. Esto es especialmente importante en entornos productivos, notebooks compartidos o plataformas cloud (como Google Colab).

Instalación

```
# En terminal: pip install seaborn  
# En Google Colab: !pip install seaborn
```

Violinplot from a wide-form dataset



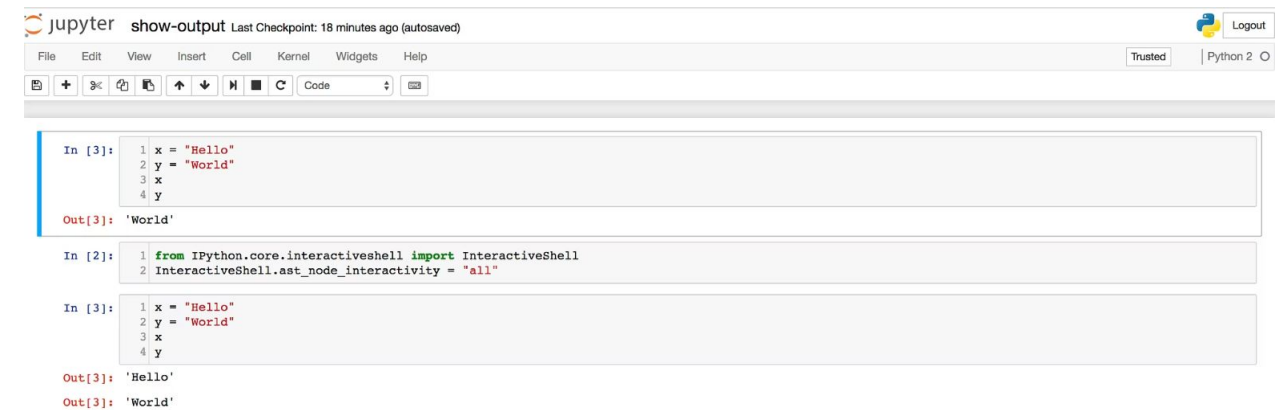
seaborn components used: `set_theme()`, `load_dataset()`, `violinplot()`, `despine()`

```
import seaborn as sns  
import matplotlib.pyplot as plt  
sns.set_theme(style="whitegrid")  
  
# Load the example dataset of brain network correlations  
df = sns.load_dataset("brain_networks", header=[0, 1, 2], index_col=0)  
  
# Pull out a specific subset of networks  
used_networks = [1, 3, 4, 5, 6, 7, 8, 11, 12, 13, 16, 17]  
used_columns = (df.columns.get_level_values("network")
```

Importación básica

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

- sns es el alias estándar para Seaborn.
- matplotlib.pyplot se suele importar para mostrar los gráficos o hacer ajustes adicionales si se desea.



```
jupyter show-output Last Checkpoint: 18 minutes ago (autosaved)
File Edit View Insert Cell Kernel Widgets Help Trusted Python 2
In [3]: 1 x = "Hello"
        2 y = "World"
        3 x
        4 y
Out[3]: 'World'

In [2]: 1 from IPython.core.interactiveshell import InteractiveShell
        2 InteractiveShell.ast_node_interactivity = "all"

In [3]: 1 x = "Hello"
        2 y = "World"
        3 x
        4 y
Out[3]: 'Hello'
Out[3]: 'World'
```

Consideraciones importantes para aprovechar la librería Seaborn

Para garantizar una correcta configuración de la librería Seaborn en tus proyectos, es fundamental seguir ciertas buenas prácticas desde el inicio. Estas no solo evitan errores comunes, sino que aseguran gráficos visualmente consistentes, entornos de trabajo estables y una mayor reproducibilidad de los análisis. A continuación, te compartimos algunas recomendaciones clave que puedes incorporar en tus flujos de trabajo.

Verificación de dependencias

Verifica que pandas y matplotlib estén instaladas antes de usar Seaborn.

Configuración de estilo

Configura el estilo gráfico con `sns.set_theme(style="whitegrid")` o similar para resultados más estéticos.

Entornos virtuales

Usa entornos virtuales para evitar conflictos de versión en proyectos grandes.

Errores frecuentes

Aunque importar Seaborn es un proceso relativamente simple, existen errores frecuentes que pueden obstaculizar la visualización de los datos o provocar fallas difíciles de rastrear. Estos errores suelen estar relacionados con dependencias mal configuradas, incompatibilidades entre versiones o errores en la preparación previa del entorno. Identificarlos a tiempo puede ahorrarte mucho esfuerzo durante la ejecución de tus análisis.



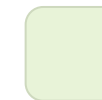
Omisión de matplotlib

Omitir la importación de matplotlib.pyplot, lo que puede impedir que los gráficos se rendericen.



Versiones obsoletas

Usar versiones muy antiguas de Seaborn que no tienen funciones actualizadas como histplot o catplot.



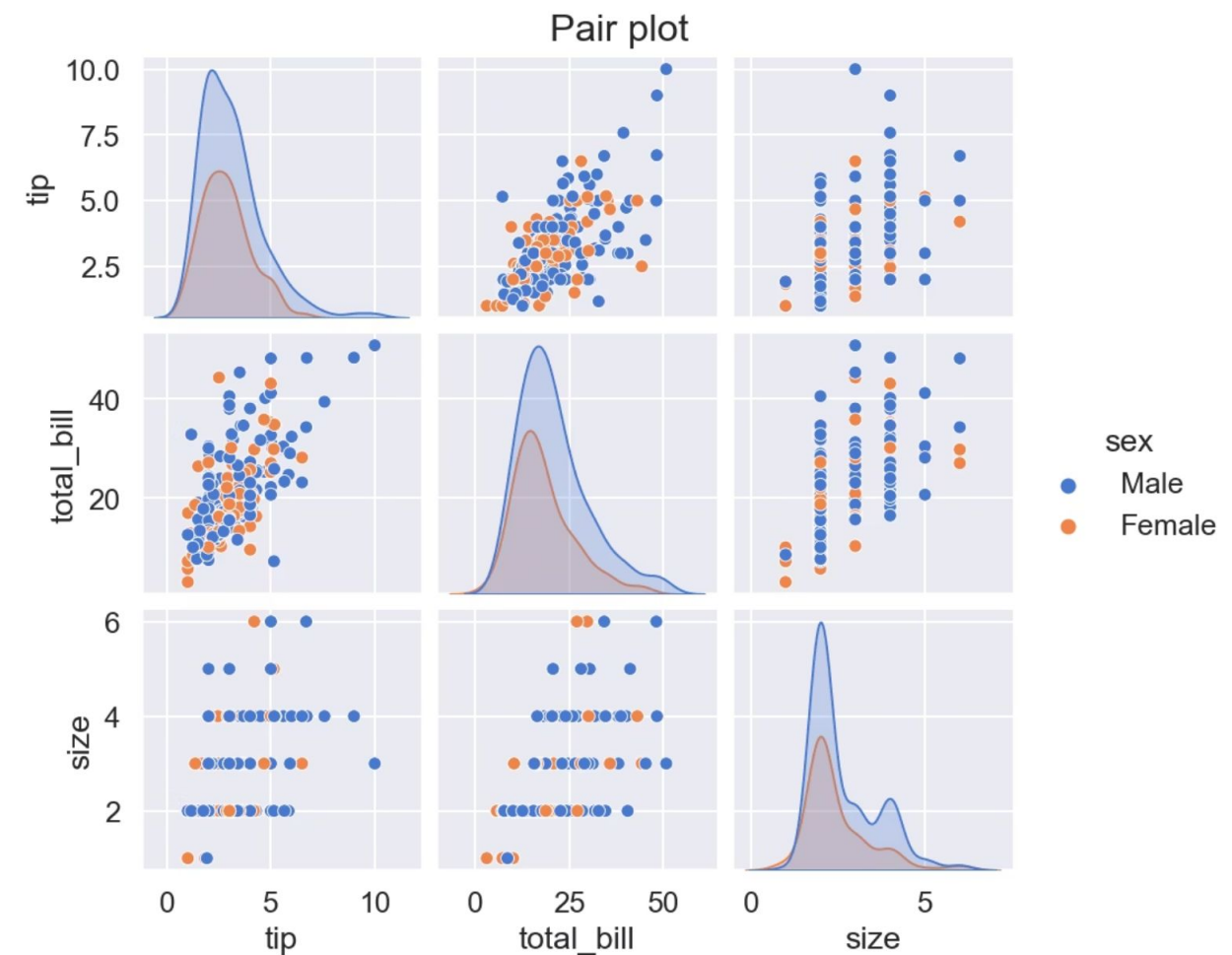
Problemas de codificación

No manejar adecuadamente la codificación UTF-8 al importar datos con caracteres especiales.

Transición a tipos de gráficos

Una vez que hemos importado correctamente Seaborn y configurado nuestro entorno de trabajo, estamos en condiciones de comenzar a crear visualizaciones. Pero ¿qué tipo de gráfico es el más adecuado según el tipo de variable o el objetivo del análisis?

En el siguiente tema exploraremos los principales tipos de gráficos que ofrece Seaborn y cómo seleccionarlos estratégicamente según el propósito de visualización.

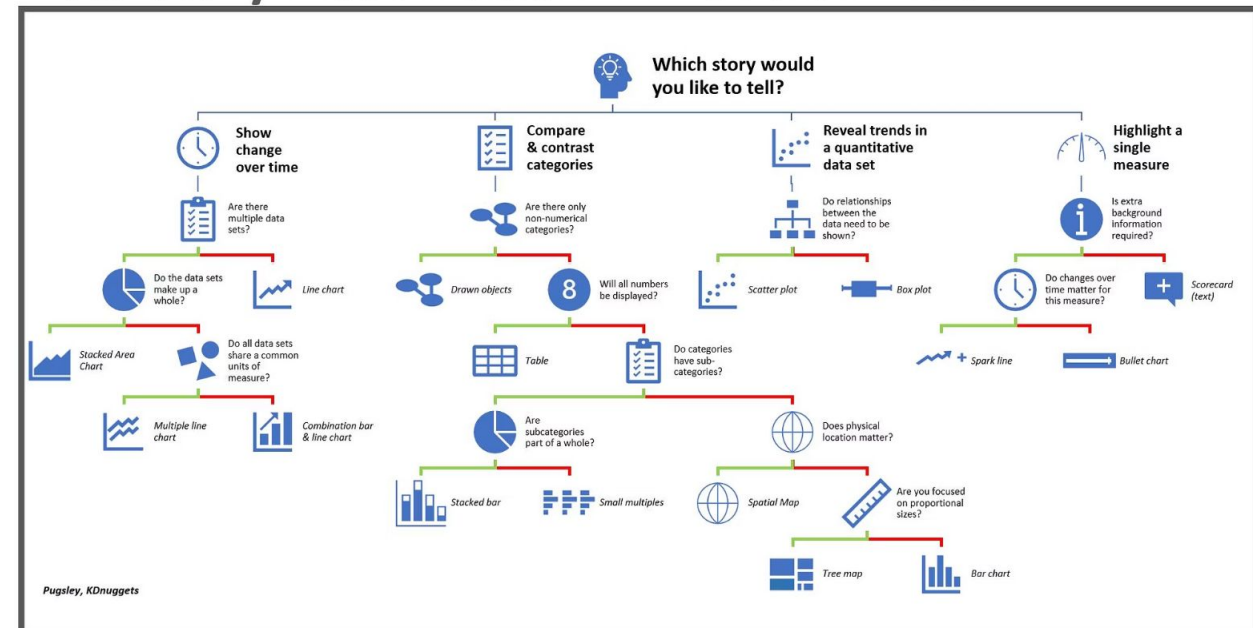


Tipos de gráficos

¿Qué tipo de gráfico debo elegir según el objetivo de mi análisis?

Una visualización efectiva no es solo estética: responde a un propósito específico. Para elegir el gráfico correcto, debemos considerar qué queremos mostrar: relaciones, distribuciones, comparaciones o composición.

Data Story Visualization: A Decision Tree



Otros gráficos

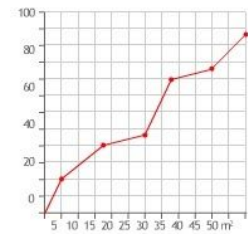


Gráfico lineal

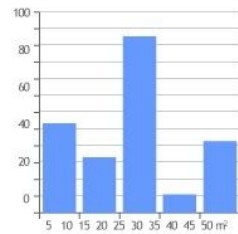


Gráfico de columna

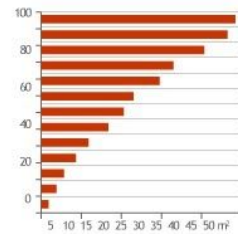


Gráfico de barras

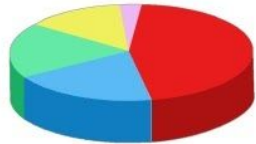
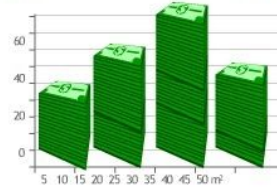


Gráfico por sectores



Pictogramas

Clasificación de gráficos según propósito

Cada tipo de gráfico tiene un propósito específico. Seaborn nos permite cubrir una amplia variedad de ellos de forma sencilla. Conocer esta clasificación nos ayudará a seleccionar la visualización más apropiada según el tipo de variable y el mensaje que queremos transmitir.

Clasificación de los gráficos más usados en Seaborn

(Parte 1)

Propósito	Tipo de gráfico	Función Seaborn
Distribución	Histograma	sns.histplot()
Distribución	Gráfico de densidad	sns.kdeplot()
Comparación	Boxplot	sns.boxplot()
Comparación	Violin plot	sns.violinplot()
Relación		

Clasificación de los gráficos más usados en Seaborn

(Parte 2)

Propósito	Tipo de gráfico	Función Seaborn
Relación	Dispersión (scatter)	sns.scatterplot()
Relación	Regresión lineal	sns.regplot()
Composición	Barras	sns.barplot()
Composición	Conteo	sns.countplot()
Multivariado	Gráfico de pares	sns.pairplot()
Multivariado	Mapa de calor	sns.heatmap()

Consideraciones al seleccionar el gráfico

Elegir el tipo de gráfico adecuado es una decisión clave en el análisis visual de datos. No se trata solo de generar una imagen atractiva, sino de representar la información de forma clara, comprensible y útil para la toma de decisiones. Una selección apropiada del gráfico, basada en el tipo de variable y el propósito del análisis, mejora significativamente la interpretación de los datos. A continuación, te presentamos algunas buenas prácticas para lograrlo:

Define el tipo de variable

Define primero qué tipo de variable quieres visualizar (cuantitativa, categórica).

Evita sobrecarga visual

Usa pairplot solo con pocas variables para evitar sobrecarga visual.

Ordena categorías

En gráficos categóricos, ordena las categorías si es posible para facilitar la lectura.

Considera la accesibilidad

Verifica que los colores usados sean accesibles para personas con daltonismo.

Errores comunes en la elección del gráfico

Aun con herramientas potentes como Seaborn, es frecuente cometer errores al elegir qué tipo de gráfico utilizar. Estos errores pueden restar valor al análisis, dificultar la comprensión de los resultados o incluso inducir a conclusiones equivocadas. Reconocer estos errores comunes es el primer paso para evitarlos y mejorar la calidad comunicativa de nuestras visualizaciones.



Complejidad innecesaria

Elegir un gráfico complejo cuando uno simple comunica mejor.



Falta de etiquetas

Omitir las etiquetas de ejes o títulos.



Colores inadecuados

Usar colores confusos o sin contraste suficiente.



Falta de adaptación

Repetir el mismo tipo de gráfico para distintos propósitos sin adaptar la visualización.

Como pudimos ver, la selección del tipo de gráfico adecuado depende del propósito del análisis y del tipo de variables que manejamos.

Seaborn ofrece un repertorio versátil de visualizaciones para explorar relaciones, comparar grupos o examinar distribuciones.

Actividad guiada: Explorando patrones de uso con visualizaciones en Seaborn



Ejercicio

Explorando patrones de uso con visualizaciones en Seaborn

Objetivo de la actividad

Aplicar técnicas básicas de visualización de datos utilizando la librería Seaborn para representar relaciones entre variables, con foco en seleccionar y utilizar correctamente distintos tipos de gráficos según el propósito analítico.

Situación problema

Eres parte del equipo de análisis de una plataforma de streaming que busca entender mejor el comportamiento de sus usuarios para mejorar la retención y personalización de contenidos. Te han entregado un archivo con datos anonimizados que incluye variables como horas de visualización por semana, edad, tipo de dispositivo, cantidad de géneros distintos consumidos y suscripción activa (sí/no).

Tu misión es crear visualizaciones que permitan comprender mejor los patrones de uso. Debes elegir los gráficos más adecuados para representar cada relación o distribución, justificar tu elección y detectar posibles patrones que podrían ayudar a segmentar a los usuarios o predecir su nivel de compromiso.

{desafío}
latam_



Información a utilizar

usuario_id,edad,horas_visualizacion,dispositivo,generos_distintos,suscripcion_activa

101,22,10.5,Smartphone,5,Sí

102,35,6.0,PC,3,No

103,28,12.0,Smart TV,6,Sí

104,41,4.0,Tablet,2,No

105,30,8.5,Smartphone,4,Sí

106,26,9.0,PC,5,Sí

107,38,5.0,Smart TV,3,No

108,45,7.5,Tablet,4,Sí

Ejercicio

Instrucciones

1. Abre tu entorno de desarrollo en Python (Jupyter Notebook, VS Code, etc.).
2. Importa las librerías necesarias:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```



Ejercicio

Instrucciones

Carga el dataset:

```
df = pd.read_csv("comportamiento_usuarios_streaming.csv")
```

Explora el contenido del DataFrame y responde: ¿Qué variables son cuantitativas? ¿Cuáles son categóricas?

Genera los siguientes gráficos, uno a la vez:

- a. Distribución de horas de visualización (histplot)
- b. Relación entre edad y horas de visualización (scatterplot)
- c. Promedio de horas por tipo de dispositivo (barplot)
- d. Comparación de géneros vistos por suscripción (boxplot)

Para cada gráfico:

- Explica por qué lo elegiste.
- Asegúrate de incluir título, etiquetas de ejes y leyenda si corresponde.
- Comenta en una celda aparte lo que observas en la visualización.



Actividad práctica autónoma: Visualizando patrones de compra en clientes



Ejercicio

Visualizando patrones de compra en clientes

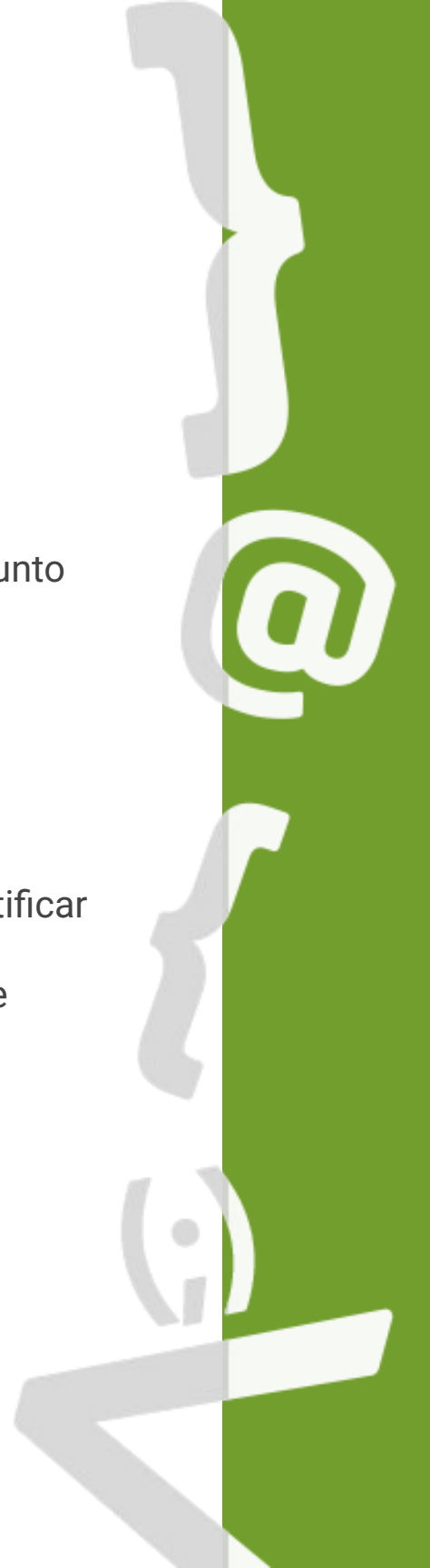
Objetivo de la actividad

Aplicar técnicas básicas de visualización de datos utilizando la librería Seaborn para representar relaciones y comparaciones en un conjunto de datos sobre clientes y sus hábitos de compra, seleccionando adecuadamente el tipo de gráfico según el propósito analítico.

Contexto

Trabajas como analista en un e-commerce minorista. Se te ha solicitado explorar los datos de compras recientes con el objetivo de identificar si existen diferencias relevantes según la edad, el género o el tipo de producto. Se espera que puedas visualizar claramente patrones que ayuden a comprender mejor el comportamiento de los distintos segmentos de clientes.

Para ello, necesitas generar gráficos apropiados que representen variables categóricas y numéricas, y redactar conclusiones claras que permitan orientar decisiones del área de marketing.



Ejercicio

Instrucciones

1. Descarga el archivo: ventas_clientes_autonomo.csv
2. Abre tu entorno de desarrollo en Python.
3. Carga el archivo y revisa las variables:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("ventas_clientes_autonomo.csv")
```



Ejercicio

Instrucciones

1

Realizar visualizaciones

- Distribución de montos de compra
(sns.histplot)
- Relación entre edad y monto de compra (sns.scatterplot)
- Comparación de monto de compra por género (sns.boxplot)
- Promedio de compra por categoría (sns.barplot)
- Proporción de compras recurrentes por categoría (sns.countplot con hue)

2

Incluir en cada gráfico

- Título
- Etiquetas de ejes
- Leyenda si corresponde

3

Escribir conclusiones

- ¿Qué patrones observas?
- ¿Qué gráfico fue más útil para responder preguntas del negocio?
- ¿Qué mejorarías en la visualización?

Resumen

Seaborn y su integración

Comprendimos qué es Seaborn, sus ventajas respecto a otras librerías, y cómo se integra con pandas y matplotlib.

Importación y configuración

Aprendimos a importar y configurar Seaborn en un entorno de trabajo.

Tipos de gráficos

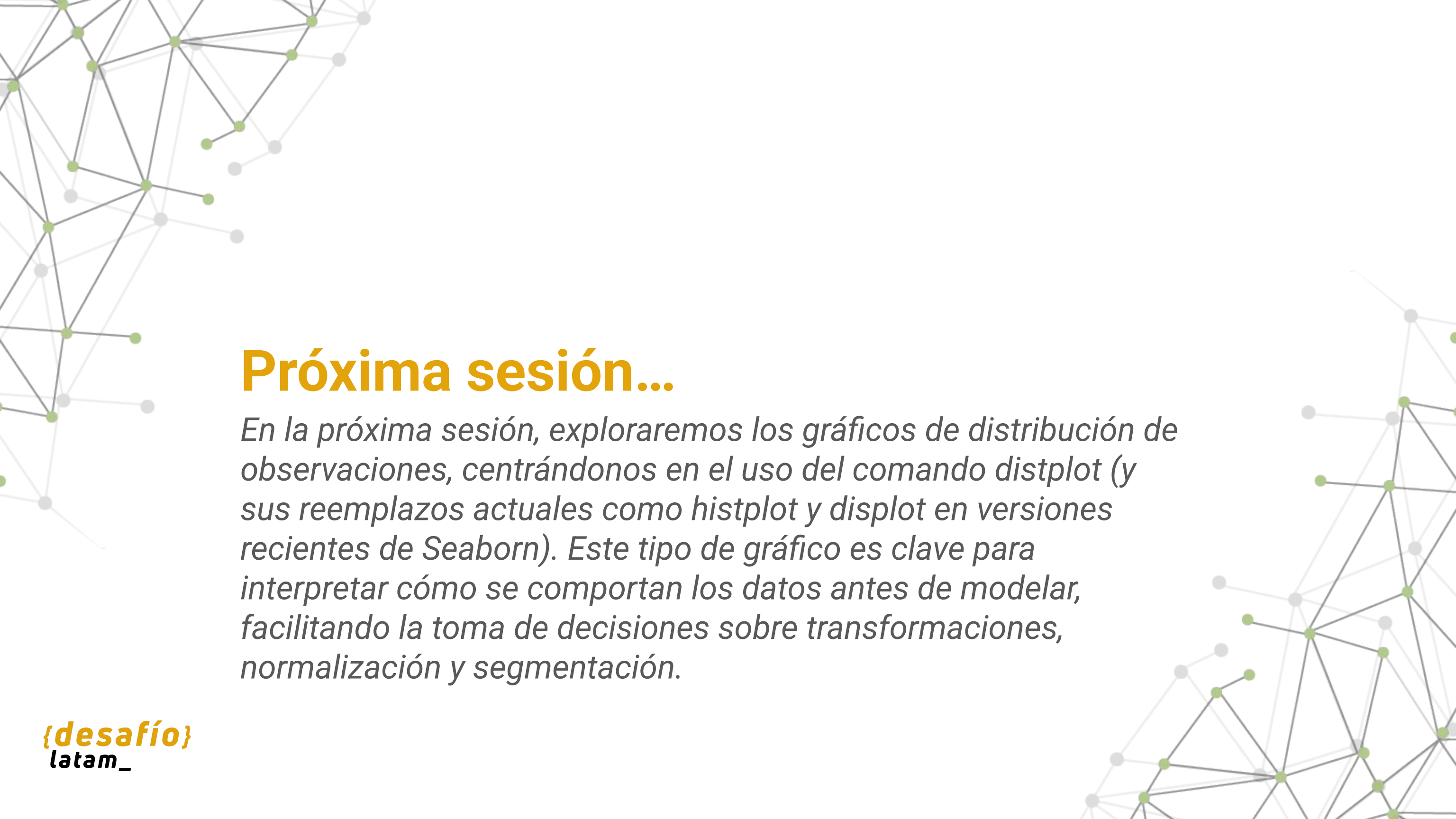
Exploramos los principales tipos de gráficos que ofrece Seaborn, clasificados según su propósito: univariado, bivariado y multivariado.

Buenas prácticas

Discutimos buenas prácticas para seleccionar gráficos apropiados y evitar errores comunes en visualización de datos.

Aplicación práctica

Aplicamos lo aprendido en una actividad práctica, construyendo gráficos reales a partir de un conjunto de datos.



Próxima sesión...

En la próxima sesión, exploraremos los gráficos de distribución de observaciones, centrándonos en el uso del comando `distplot` (y sus reemplazos actuales como `histplot` y `displot` en versiones recientes de Seaborn). Este tipo de gráfico es clave para interpretar cómo se comportan los datos antes de modelar, facilitando la toma de decisiones sobre transformaciones, normalización y segmentación.